

Speaker Notes

SLIDE 1 Title slide · *Title · 20 sec*

My topic today is a comparison of three software architecture styles from Richards and Ford's Fundamentals of Software Architecture: microkernel, service-based, and microservices — chapters 13, 14 and 18.

Rather than describe each one in isolation, I'm going to compare them side by side on the same criteria, so by the end you'll have a working framework for choosing between them, not just three definitions.

SLIDE 2 Agenda · *Agenda · 30 sec*

Here's the shape of the talk. I'll spend a few minutes on why this decision matters and how I'll compare the styles fairly, then walk through each architecture in turn — microkernel, service-based, microservices — with a real diagram, its trade-offs, and a real-world example for each. After that we'll put all three head-to-heads, talk about how to choose between them, and close with takeaways before questions.

SLIDE 3 Why the style decision matters · *Framing · 60 sec*

Richards and Ford's first law of software architecture is that everything in software architecture is a trade-off. If you take one thing from this slide, take that — not as an evasion, but as an honest starting point for everything that follows.

The architecture style decision is unusual because it fixes deployment boundaries, data ownership, and team structure all at once. It's the decision that's hardest and most expensive to reverse, so it deserves more rigor than 'what's popular right now.' Three ideas sit underneath that: structure has to follow the constraints of the problem, not the other way around; power has a price — Richards uses the image of a Ferrari stuck in rush-hour traffic, capability you never use is still something you paid for; and there's no ranking of styles, only a fit between one style and one specific problem.

SLIDE 4 How we will compare them · *Framework · 45 sec*

So that this isn't just my opinion versus another architecture's fans, I'm scoring all three styles on the same card, straight from the book: partitioning type, number of architecture quanta, and eleven architecture characteristics — things like deploy ability, scalability, cost, and so on.

One star means the characteristic is poorly supported by that style; five stars means it's one of the style's defining strengths. These ratings describe tendencies of a style in general, not a guarantee about any specific system you build — a well-built service-based system can absolutely outperform a badly built microservices one.

SLIDE 5 Why these three styles · *Scope · 45 sec*

Before diving in, it's worth explaining why these three and not, say, layered or event-driven architecture. These three sit on a single axis: service granularity, from one big deployable unit in microkernel, to a handful of coarse services in service-based, to many fine-grained services in microservices. Because they differ only in that one variable, every difference in cost, scalability, and complexity we're about to see is directly attributable to granularity — not to some unrelated design idea getting mixed in. That's what makes the comparison clean.

SLIDE 6 Microkernel: core system plus plug-ins · *Microkernel · 75 sec*

Microkernel, sometimes called plug-in architecture, has a minimal core that handles the general processing flow, and everything volatile or customer-specific lives in independent plug-in components.

The core contains only the happy path — the minimum needed to run — while plug-ins absorb variation: per-client rules, per-jurisdiction regulations, optional features. It's usually one monolithic deployment sharing a single database, and it's the only style in the book that can be partitioned by both domain and technical concern at once.

The example on screen is deliberately mundane: without plug-ins, every new device type becomes another 'else if' branch and the core's complexity grows without bound. With a registry and plug-ins, adding a new device is just adding a component and a registry entry — the core code itself never changes.

SLIDE 7 Microkernel topology · *Microkernel · 45 sec*

Walking the diagram: the user interface talks to the core, the core consults its plug-in registry, and the core invokes the plug-in it needs. The core also owns the shared database — plug-ins normally don't touch it directly. The important detail here is that even if plug-ins are reached remotely over REST or messaging, the whole application still counts as a single architecture quantum, because every request funnels through the one core.

SLIDE 8 Microkernel mechanics · *Microkernel · 60 sec*

A few mechanics worth naming. The registry tells the core which plug-ins exist and how to reach them — it can be as simple as a hash map, or as sophisticated as an external discovery tool like Consul.

Contracts standardize how the core talks to a whole domain of plug-ins, so third-party plug-ins just need an adapter rather than special case code in the core. Binding time matters too: compile-based plug-ins are simpler but need full redeploy, while runtime plug-ins can be swapped in and out live — that's how every IDE extension marketplace works.

The one row I'd flag specifically is remote plug-ins. Reaching a plug-in over the network buys decoupling and asynchrony, but the moment you do that you've quietly turn part of a monolith into a distributed system with all distributed computing's usual failure modes.

SLIDE 9 Microkernel: trade-offs · *Microkernel · 45 sec*

Here's the score card. The headline pairing is cost and scalability: five stars for overall cost the cheapest of the three styles against one star for scalability, elasticity, and fault tolerance.

A single monolithic deployment scales and fails as one unit. Simplicity is also strong, at four stars, because there's no network between components to reason about. The takeaway: choose microkernel when the product needs to be extended, not when it needs to be scaled.

SLIDE 10 Case study: Visual Studio Code · *Microkernel · 60 sec*

For a real example, Visual Studio Code is a textbook microkernel. The core is a deliberately small text editor and workbench; almost everything else — language support, debuggers, linters, even AI assistants — is an extension. Extensions declare themselves through a manifest of contribution points and activation events — that manifest is literally the plug-in registry from the book. And extensions run in a separate process from the editor UI, so a badly behaved extension can't freeze the whole editor — that's a direct, practical answer to microkernel's usual fault-tolerance weakness.

The same shape shows up constantly in business software: insurance claims processing, where the filing workflow is stable, but rules differ by jurisdiction, or tax software, where the summary return is the core and each tax form is a plug-in.

SLIDE 11 Service-based: the pragmatic middle ground · *Service-based · 60 sec*

Service-based architecture is best understood as a hybrid of microservices that keeps most of the modularity while dropping most of the bill. It's a distributed, macro-layered structure: a separately deployed UI talking to coarse-grained domain services, usually sharing one database.

A typical system has four to twelve of these services — averaging around seven — and critically, services don't call each other. Each one is self-contained, which means you keep ordinary ACID transactions and don't need sagas or distributed-transaction machinery.

The reason this style exists: many organizations want faster, safer deployments and clear domain ownership, but they don't have — and can't afford — the automation and operational maturity that fine-grained microservices demand.

SLIDE 12 Service-based topology · *Service-based · 45 sec*

In the diagram, the UI optionally goes through an API layer or gateway, then reaches each domain service directly — Quoting, Receiving, Assessment, and so on.

Each service is internally layered with its own facade, business logic, and persistence code. The one detail to notice is what's missing: there are no arrows between the services themselves. That absence of service-to-service chatter is the defining feature of this style, and it's what keeps failures contained to a single domain.

SLIDE 13 Variants · *Service-based · 45 sec*

Service-based is really a family of topologies, not one fixed shape. Three independent dials: the UI can be one monolith, split by domain, or one per service; the database can be fully shared, grouped by domain, or fully split; and the API layer can be nothing at all, a simple locator, or a full gateway. Each dial can move independently, which is exactly why this is the most flexible of the three styles — you buy only the decoupling your specific problem needs.

SLIDE 14 Service-based: trade-offs · *Service-based · 45 sec*

The honest headline here is that there are no five-star ratings anywhere on this scorecard — and no one-star ratings. Four stars across deploy ability, testability, modularity, fault tolerance, reliability, and cost.

That flat, balanced profile is the whole argument for this style: it doesn't dominate at any one thing, but it's never bad at anything either. If a system needs a genuine five-star extreme — massive scale, say — this style won't deliver it, and that's a legitimate reason to look elsewhere.

SLIDE 15 Case study: Amazon Prime Video · *Service-based · 75 sec*

In 2023 Amazon's Prime Video team published an unusually candid account of moving one component — audio and video quality monitoring — away from fine-grained serverless microservices toward a single coarser service.

Before: media conversion, defect detection, and orchestration were separate serverless components, coordinated by AWS Step Functions, passing video frames through an S3 bucket. That racked up per-transition charges and heavy storage costs and even hit account service limits.

After: everything runs in a single process, with data handed off in memory instead of through S3, and the service is simply cloned across clusters when more capacity is needed. The result was a 90 percent reduction in infrastructure cost, with higher scale.

I want to be precise about what this proves. It's not that microservices are wrong Amazon said so explicitly. It shows that when components are chatty and pass large payloads, network and orchestration overhead dominate the bill, and coarser boundaries eliminate both. That's the argument for service-based architecture in one case study: pick the granularity your workload justifies, then stop.

SLIDE 16 Microservices: decoupling taken to its limit · *Microservices · 60 sec*

Microservices takes decoupling to its logical extreme. The organizing idea is the bounded context: each service models one domain and owns everything it needs its code, its subcomponents, and critically, its own database schema.

It's distributed by definition: every service runs in its own process, usually a container, which removes resource contention but means every call now crosses the network and typically passes a security check. It's domain-partitioned to the extreme the most distinct architecture quanta of any style in the book — and it explicitly prefers duplication over coupling: sharing a common Address class between two microservices is treated as an anti-pattern, not a saving.

Data isolation follows the same logic: no shared schema, no database used as an integration point. One service is the named source of truth for a fact, and others replicate what they need.

SLIDE 17 Microservices topology · *Microservices · 45 sec*

In the diagram, requests come in through a thin API layer — routing and a single-entry point only, deliberately never a place to put business logic — and fan out to independent services, each with its own database.

The one thing microservices does reuse is operational tooling, and it does that through a sidecar pattern rather than a shared library: logging, monitoring, and circuit breaking get injected alongside each service, so you get consistent operations without coupling every service to a shared release schedule.

If the API layer starts accumulating orchestration logic, you've quietly rebuilt the enterprise service bus that microservices was originally a reaction against.

SLIDE 18 Granularity and transactions · *Microservices · 60 sec*

Martin Fowler's line captures this well: the term microservice is a label, not a description — size alone doesn't tell you if a boundary is right.

Three tests for a good boundary: does the service serve one clear purpose; do entities that change together live inside the same service, since transactions are a strong signal for where a boundary belongs; and if two services can't function without constant back-and-forth chatter, that's a sign they should be merged, not split.

The common failure modes: services cut too small, which forces communication links back between them just to get useful work done; modeling one service per database table, which produces a distributed CRUD layer rather than a real domain architecture; and reaching for two-phase commit across services instead of accepting eventual consistency with sagas.

SLIDE 19 Microservices: trade-offs · *Microservices · 45 sec*

This is the mirror image of the microkernel scorecard: five stars for scalability, elasticity, evolutionary change, and modularity, against one star for both overall cost and simplicity.

Performance is worth calling out specifically — only two stars, because every request can involve many network calls and a security check at each hop. The headline: you're buying scale and evolvability, and you're paying for it in cost, simplicity, and latency.

SLIDE 20 Case study: Netflix and Uber · *Microservices · 75 sec*

Netflix is the style's proving ground: hundreds of independently deployable services behind one streaming experience, each owned end to end by its team. But resilience there was engineered deliberately — circuit breakers, an edge gateway, service discovery, chaos testing in production. The lesson is that the tooling isn't optional overhead; it's what makes the style survivable at all.

Uber is the most interesting cautionary tale. By 2020 they were running roughly 2,200 critical microservices after scaling from hundreds to thousands of engineers. The pain wasn't scale — it was complexity: one root-cause investigation reportedly required working through around 50 services across 12 teams.

Uber's response was Domain-Oriented Microservice Architecture, or DOMA: group related services into domains, put a gateway in front of each domain so consumers depend on the domain rather than its internals, and constrain dependency direction between domains.

Here's the connection I want to land: a domain gateway fronting a group of cohesive services is, structurally, a step back toward coarse-grained domain services — the service-based idea, rediscovered at scale by the industry's most committed microservices adopter.

SLIDE 21 Head to head · *Comparison · 45 sec*

Here's every characteristic for all three styles on one table. I won't read it line by line — I'll just point at three rows.

Microkernel and microservices are near-mirror images on cost and scalability: five and one, then one and five. Service-based has no ones and no fives anywhere — its whole pitch is the absence of extremes. And elasticity is the cleanest single discriminator in the whole table: one, two, five. If your workload has spiky, uneven load across domains, that row alone can decide the argument.

SLIDE 22 The shape of each trade-off · *Comparison · 45 sec*

Same data, as a chart, to make the shapes easier to see immediately. Note that I've inverted cost here, so a taller bar means cheaper — microkernel wins outright on that measure, and microservices is the most expensive style in the book.

Notice how closely service-based tracks microservices on deploy ability, testability, and fault tolerance, then diverges sharply on elasticity and scalability. And performance is genuinely a weak point for microservices, at two stars — not the strength its popularity might suggest.

SLIDE 23 How to choose · *Decision · 60 sec*

Richards and Ford don't rank these styles — they give inputs to a decision. I've turned that into four questions, meant to be asked by a stakeholder, not run as an automatic flowchart. The order matters: the first question comes before the others.

First: does the whole system genuinely need one set of characteristics? If yes, a monolith is a legitimate, defensible answer — you don't need to distribute anything. If no, you're in distributed territory and the added cost follows from that, not from a style choice yet.

Second: is the dominant requirement customization or extensibility, like per-jurisdiction rules or a product installed at customer sites? That points straight at microkernel. Third: can the domains share one database without fighting over it? If yes, service-based keeps your ACID transactions and a much smaller bill. Fourth: do different domains genuinely need different scales and elasticity? Only then does per-service deployment and data isolation earn its cost — that's microservices.

Underneath all of this is domain and architecture isomorphism: a highly coupled problem resists a highly decoupled architecture. A multi-page form where every page depends on the last does not want to be microservices, no matter how fashionable that is.

SLIDE 24 Evolution, anti-patterns, course context · *Synthesis · 60 sec*

A common and defensible migration path runs from monolith to modular monolith, to service-based, to microservices — but every arrow on that path should be justified by an actual requirement, not by momentum. Service-based is a legitimate destination, not just a waypoint; many organizations that stop there are better served than the ones that keep going.

Four anti-patterns worth naming: the distributed monolith, where services must be deployed together and you pay all the cost of distribution for none of the independence; the entity trap, one service per database table; architecture by implication, writing code before naming the style; and what I'd call Groundhog Day — re-litigating the same architecture decision repeatedly because nobody wrote down why it was made.

And I want to explicitly connect this to the rest of the course: domain-driven design gives us the vocabulary that microservices boundaries are drawn in; the microservices patterns and migration topic picks up exactly where I'm stopping, with sagas and the strangler fig pattern; API design is where these granularity decisions become visible as actual contracts; and event-driven architecture is the asynchronous alternative to the request-response coupling I've assumed throughout.

SLIDE 25 Key takeaways · Conclusion · 60 sec

Four takeaways. One: there is no best style, only a best fit — every one of these three is optimal somewhere and indefensible somewhere else, and the architect's real job is correctly naming which situation you're in.

Two: granularity is the hidden variable underneath everything — coupling, cost, latency, and deployment independence all move together as services get smaller. You cannot buy one of those without paying for the rest.

Three, and this is the one I'd underline: service-based deserves more credit than it gets. No extremes, no five-star ratings anywhere — and for a large share of real business applications, it's exactly the right answer, even though it gets far less attention than microservices.

Four: cost and complexity are themselves architecture characteristics, not afterthoughts. Both Prime Video and Uber retreated from fine granularity, and in neither case was it a technical failure — both were complexity and cost failures.

SLIDE 26 References · Conclusion · 60 sec

“On this final slide, I would like to briefly highlight the main references used in our work.

Our primary source is *Fundamentals of Software Architecture* by Mark Richards and Neal Ford, particularly the chapters covering microkernel, service-based, microservices, and architecture style selection.

We also referred to several important books, including Sam Newman's *Building Microservices*, Richards and Ford's *Headfirst Software Architecture*, and Vernon's *Domain-Driven Design Distilled*. Martin Fowler and James Lewis's work on microservices was also useful for understanding the key principles of microservice architecture.

For industry examples and case studies, we considered real-world material from Amazon Prime Video, Uber, Netflix, and Microsoft. These sources helped connect the theoretical concepts with practical implementations.

Together, these references provided both the theoretical foundation and real-world perspective for our analysis of software architecture.”

Questions & Answers

General framework

Q Why did you choose these three architectures specifically, rather than comparing against layered or event-driven?

A Because they sit on a single axis — service granularity — going from one deployable unit to a handful of coarse domain services, to many fine-grained services. Since only granularity changes between them, every difference in cost, scalability, and complexity is attributable to that one variable rather than to an unrelated design idea.

Q Is there an objectively “best” architecture among the three?

A No, and the book is explicit about that — there's no ranking, only fit. Each of the three is the correct choice under some combination of domain requirements, data architecture, team maturity, and organizational constraints, and indefensible under others.

Q How reliable are these star ratings? Aren't they subjective?

A They're the authors' own scorecards from the book, so they're a documented industry consensus rather than my personal judgment, but you're right that they describe a typical implementation of a style, not a guarantee about any specific system. A very well-engineered service-based system can outperform a badly built microservices system on any given characteristic.

Q Can a style's weak characteristic be fixed with better engineering?

A Only by trading it against another characteristic. For example, you can improve microkernel's fault tolerance by making plug-ins remote and independently deployable but that costs you simplicity and turns part of the monolith into a distributed system with its own new failure modes. The ratings are closed, not a checklist you can max out.

Q What is an architecture quantum, exactly, and why does it matter so much in this comparison?

A It's an independently deployable unit of software with high functional cohesion, including its own data. It matters because the number of quanta is really what separates these three styles at the deepest level: microkernel is always one quantum no matter how it's implemented, service-based can be one or several, and microservices deliberately maximizes the number of quanta, which is where both its benefits and its costs come from.

Microkernel

Q What's the actual difference between a microkernel plug-in and a normal library or module?

A A plug-in is discovered and invoked through a registry at runtime or load time, following a standard contract the core understands, rather than being compiled directly against by name. That's what lets you add or remove a plug-in without touching or redeploying the core — you can't do that with an ordinary imported library.

Q If the core still must know about every plug-in's contract, hasn't it just centralized the coupling instead of removing it?

A There's real coupling to the contract, yes, but the core only needs to know one standard contract per plug-in domain, not the internals of every individual plug-in. Adding a new plug-in that honors the existing contract requires zero changes to the core — that's the isolation the style is selling, not zero coupling.

Q Why would anyone choose microkernel over a service-based architecture, if service-based has better fault tolerance and scalability ratings?

A Because those two ratings aren't what the requirement calls for in a microkernel-shaped problem. When the dominant need is deep customization or per-customer extensibility — a product installed at many customer sites, or rules that vary by jurisdiction — microkernel's isolation of volatility is worth more than service-base's better scaling story, and it costs far less.

Q Isn't the core system in microkernel just a monolith with extra steps?

A In terms of deployment, essentially yes — it's usually one monolithic deployment sharing one database, which is exactly why it inherits the monolith's cost and simplicity advantages along with its scalability weaknesses. What separates it from an undifferentiated monolith is that volatility is deliberately pulled out into plug-ins, so the core's own complexity stays bounded as the product grows.

Service-based

Q How is service-based architecture different from microservices, if both use services?

A Three concrete differences: service-based uses four to twelve coarse, domain-sized services rather than many fine-grained ones; it normally shares a single database rather than giving each service its own; and services don't call each other, so there's no interservice choreography or orchestration to manage. It keeps the modularity benefit of services while avoiding almost all of the distributed systems complexity microservices takes on.

Q If service-based shares one database, doesn't that just recreate the tight coupling problem microservices was designed to solve?

A It reintroduces schema-level coupling, yes — that's the honest trade-off, and it's the reason the style tops out at four stars rather than five on several characteristics. But it buys back ACID transactions and referential integrity for free, which is exactly what you lose the moment you split the database, so which coupling you'd rather live with depends on whether your domains are transactionally intertwined.

Q You said services don't call each other in service-based architecture — what happens when a business process genuinely needs two domains, like checkout needing both inventory and payment?

A That's typically handled by pushing orchestration up into the API layer or the UI, which calls the domain services it needs in sequence, rather than having the services call each other directly. Because they usually still share one database, the whole operation can often still be wrapped in one ordinary transaction, which is harder to do once services own separate databases.

Q Isn't service-based architecture just microservices done badly, or an excuse to avoid doing microservices properly?

A No — the book presents it as a legitimate, distinct style with its own scorecard, not a failed or partial microservices implementation. The Prime Video case study is a real example of an organization deliberately moving toward this coarser granularity after finding fine-grained services too expensive for that specific workload, which is the opposite of it being a fallback for teams who couldn't manage microservices.

Microservices

Q How do you decide how small a microservice should be? Is there a rule of thumb?

A The book gives three tests rather than a size rule: does the service serve one clear, cohesive purpose; do the entities that must change together in one transaction live inside it; and can it function without constant chatter to other services. If a proposed split fails the transaction or chatter test, the services are probably too small and should be merged back.

Q With each microservice having its own database, how do you ever run a transaction across two services, like debiting one account and crediting another?

A You generally don't use a single ACID transaction across services — the standard pattern is a saga, a sequence of local transactions coordinated through events or orchestration, with compensating actions if a later step fails. You accept eventual consistency in exchange for not copying two services' databases together.

Q If microservices is the most expensive and complex style, why is it still the most popular one in industry right now?

A Partly genuine fit — companies like Netflix have workloads with real, uneven, extreme scaling needs where the five-star scalability and elasticity ratings are worth the cost. But partly it's also fashion outrunning fit, which is exactly why Uber's DOMA story is worth telling: even a company with deep microservices expertise found that decomposition had gone further than its actual requirements justified.

Q Doesn't the sidecar pattern you showed in the topology diagram add its own overhead and complexity?

A Yes, a service mesh of sidecars itself is a nontrivial operational system to run and monitor, and that overhead is folded into microservices' one-star simplicity and one-star cost ratings. The trade being made is that this cost is centralized and consistent across every service, rather than each team reinventing its own logging, discovery, and circuit-breaking methods, which would be worse.

Case studies

Q Did Amazon abandon microservices after the Prime Video result?

A No, and their own published article is explicit about that — this was one component, audio and video quality monitoring, within one product. Amazon continues to run large parts of its business on fine-grained microservices; the point of the case study is that the right granularity is chosen per workload, not applied as a company-wide dogma in either direction.

Q Is the 90% cost figure from Prime Video a fair comparison, or could it be explained by something other than the architecture change?

A It's a fair question — the published figure bundles the architectural change together with the move off several separate serverless components with per-invocation billing, so some of the saving is specifically about eliminating chatty, billed transitions rather than granularity in the abstract. I'd frame it as strong evidence for 'coarser is cheaper when components are chatty,' not as a universal 90% number to expect elsewhere.

Q You said Uber ran about 2,200 microservices — is that still accurate today?

A That figure is from Uber's own 2020 engineering blog post announcing DOMA, so it's a snapshot from that point in their growth, not a current number — the count has almost certainly changed since. What's durable is the pattern they described: complexity outpacing team growth, which is the part of the story that generalizes.

Q Is DOMA the same thing as service-based architecture, or are you overstating the similarity?

A They're not identical — DOMA still has many fine-grained services behind each domain gateway, so the underlying quanta count is still high, unlike service-base's four-to-twelve coarse services. What I'm claiming is narrower: structurally, a gateway that fronts a group of related services so that outside consumers depend on the domain rather than individual services is the same coarsening instinct that defines service-based architecture, applied inside a microservices system rather than as a replacement for it.

Q Is Visual Studio Code really a good example of microkernel, given it's a desktop application and not the kind of enterprise system this course is about?

A The style is defined by structure, not by domain — a small core plus independently loadable plug-ins behind a registry and contract, which VS Code implements almost exactly as the book describes it. I included it because it's a system everyone in the room has used, which makes the abstract plug-in-registry-contract vocabulary concrete; the insurance and tax examples in the book are the directly enterprise-flavored parallels.

Decision-making & synthesis

Q Your four decision questions look sequential — what happens if a system answers 'yes' to more than one, like needing both customization and extreme scale?

A Real system often does span more than one need, and that's normal — the four questions are meant to surface which forces are in tension, not to output a single answer automatically. In practice that often means a hybrid: for example, a service-based core for most domains, with the one or two components that genuinely need extreme elasticity carved out as microservices.

Q You mentioned domain and architecture isomorphism — can you give a concrete example of a problem that resists being decoupled?

A A multi-step wizard or form where each page's logic depends on validated state from the previous page is a good example — splitting that across independently deployed services would mean constantly passing partial state back and forth over the network, which is slower and more fragile than keeping it in one process. The workflow's own coupling should be reflected in the architecture's coupling, not fought against.

Q If you had to build one of these three for a real project, which would you pick and why?

A It depends entirely on the project's specific requirements, which is the whole argument of this talk — but if I had to generalize, most business applications I've seen described in coursework and industry writing fit comfortably inside service-based architecture's balanced profile and only reach for microservices once a specific component's scale or elasticity need is proven, not assumed.

Q How would migrating from one of these styles to another work in practice? Is it a rewrite?

A Not usually a full rewrite — the common, defensible path goes from monolith to modular monolith to service-based to microservices, each step justified by a specific new requirement rather than done wholesale. The strangler fig pattern, covered in the microservices patterns and migration topic later in the course, is the standard technique: route traffic for one capability at a time to a new service while the rest of the system keeps running unchanged.

Methodology & scope

Q What did you use as sources beyond the assigned chapters?

A The assigned chapters, 13, 14, and 18, plus chapter 19 on choosing an architecture style for the decision framework, were the backbone. Beyond the book, I used Uber's and Amazon's own published engineering blog posts for the case studies, and Microsoft's public extension API documentation for the VS Code example, all of which are listed on the references slide.

Q Why include Netflix and Uber both, rather than going deeper on just one microservices example?

A They make opposite points with the same style: Netflix shows what microservices looks like when it's engineered well and the requirement genuinely justifies it, while Uber shows what happens when decomposition outpaces the organization's ability to manage it. Using only one would have made microservices look either uniformly good or uniformly bad, which isn't the honest picture.

Q How confident are you in the specific numbers you cited, like the 4–12 range for service-based services or the star ratings?

A The star ratings and the four-to-twelve service count are both stated directly in the book's own chapters, so I'm confident in those as the authors' documented position. The case-study numbers — Uber's service count, the Prime Video percentage, are self-reported by those companies in public engineering posts, which I've treated as illustrative evidence rather than independently verified fact.